

Candidate Profile Breakdown

Education: B.Sc. in CSE, Ahsanullah University of Science and Technology (AUST)

Current Experience: Software Developer @ XORGeek (Aug 2023 – Present)

Production Systems: RunMate Club, Koyama Shokai, Pet App, KEIAI, Izumi, Ginsen

Research: 3 Publications (Bengali Food ML, IEEE Face BMR DL, Drought Forecasting)

Flutter & Dart

Angular & TypeScript

AWS Serverless (Lambda, DynamoDB)

MySQL & PostgreSQL

SSLCommerz Payment Gateway

RBAC & Session Security

Docker & CI/CD

REST API Architecture

1Questions on Your Work Experience & Projects

Q1. You integrated SSLCommerz in RunMate Club. Walk us through a secure payment gateway handshake from user checkout to database commit.

Structured Model Answer:

1. Initiation: Client clicks Pay \$→ Backend initiates transaction with SSLCommerz API (passes `store\_id`, `total\_amount`, `tran\_id`, `currency`, `callback\_urls`).

2. Redirect & Payment: User is redirected to SSLCommerz Hosted Gateway or popup and authorizes payment (Card/Rocket/bKash/Nexus).

3. IPN (Instant Payment Notification) & Webhook: SSLCommerz sends a server-to-server POST request (`IPN`) with `val\_id` and `tran\_id`.

4. Validation & Verification: Backend queries SSLCommerz validation endpoint using `val\_id` to verify genuine success, currency, and exact amount matches.

5. Idempotency & Database Commit: Use an atomic database transaction (`BEGIN ... COMMIT`) to update invoice status from `PENDING` to `SUCCESS`. If `tran\_id` is already processed, reject duplicates.

Key Bank Keyword: Idempotency Key and Server-Side IPN Validation (Never trust client-side return URLs for ledger updates).

Q2. In Koyama Shokai, you designed a 4-tier Role-Based Access Control (RBAC). How does this translate to a Banking Maker-Checker system?

Answer Focus:

• Maker-Checker Principle: Core regulatory banking rule where no single officer can authorize a transaction alone. A "Maker" creates the transaction (Status: `PENDING\_APPROVAL`), and a "Checker" (Manager/Supervisor) verifies and approves it.

• Implementation: Angular route guards handle UI visibility, but strict security is enforced at the API gateway / Lambda layer via JWT claims / Session context checking both user role, branch ID, and transaction threshold limits.

• Tamper-proof Audit Trail: Every action records `maker\_user\_id`, `checker\_user\_id`, `timestamp`, `ip\_address`, and before/after state diff in an append-only audit ledger.

Q3. DBBL operates high-volume mobile apps (Rocket, NexusPay). As a Flutter developer, how do you harden mobile banking applications against attacks?

- **Root / Jailbreak Detection:** Block app execution on compromised devices (using libraries like ``flutter_jailbreak_detection`` or native integrity APIs).
- **SSL Certificate Pinning:** Hardcode SSL certificate public keys in the app to prevent Man-in-the-Middle (MitM) proxy attacks (e.g. Charles / Burp Suite).
- **Screen Capture & Overlay Protection:** Set ``FLAG_SECURE`` in Android native window manager to prevent screenshots, screen recording, and tapjacking overlays.
- **Encrypted Local Storage:** Store session tokens only in Android Keystore / iOS Keychain (using ``flutter_secure_storage``), never in ``SharedPreferences`` or plain SQLite.

2 Banking & FinTech Domain Mastery (Must-Know for DBBL)

Q4. Explain ACID properties in the context of an inter-account fund transfer. What happens if the server crashes mid-way?

Property	Banking Meaning	Example in ₳10,000 Transfer (A $\rightarrow$ B)
Atomicity	All or nothing execution.	Debit from A and Credit to B must both succeed. If server fails after debiting A, entire transaction rolls back (<code>`ROLLBACK`</code>).
Consistency	Preserves database invariants & balance rules.	Account A cannot go below minimum balance or zero; total sum ($A + B$) remains unchanged before & after.
Isolation	Concurrent transactions do not interfere.	If Account A tries to withdraw while a transfer is happening, row locking prevents double-spending.
Durability	Once committed, changes survive system failure.	Transaction log (WAL / Redo Log) is written to non-volatile disk before returning success response.

Q5. What is the difference between NPSB, BEFTN, and RTGS in Bangladesh's banking ecosystem?

System	Full Form	Settlement Speed	Typical Use Case / Limits
NPSB	National Payment Switch Bangladesh	Instant / Real-time (24/7)	Interbank ATM withdrawals, POS card payments, Internet Banking Fund Transfer (IBFT) up to ₳3,00,000.
BEFTN	Bangladesh Electronic Funds Transfer Network	Batch / Next Business Day	Salary disbursements, dividend payments, large batch account-to-account transfers.
RTGS	Real-Time Gross Settlement	Immediate / Gross (Working hours)	High-value transactions (Minimum ₳1,00,000, typically corporate, multi-crore settlements).

Q6. What is ISO 8583? Why is it vital to DBBL's ATM and POS Switch network?

ISO 8583 is the international standard messaging protocol for Card-Originated Financial Transactions. When a customer inserts a DBBL Nexus or VISA card into an ATM, the ATM converts the PIN, track data, and withdrawal amount into an ISO 8583 formatted message:

- **MTI (Message Type Identifier):** e.g., `0100` (Authorization Request), `0110` (Authorization Response), `0200` (Financial Transaction Request).
- **Bitmap:** Identifies which Data Elements (DEs) are present in the packet.
- **Data Elements:** DE-2 (Primary Account Number), DE-3 (Processing Code), DE-4 (Transaction Amount), DE-11 (System Trace Audit Number - STAN), DE-52 (PIN Data).

3 Core CS, Databases & High Availability

Q7. Why do core banking systems strictly use Relational Databases (Oracle / PostgreSQL / SQL Server) instead of NoSQL (MongoDB / DynamoDB)?

- **Strict ACID Guarantees:** Core ledgers cannot tolerate "eventual consistency" where balance updates might lag across nodes. Relational databases provide strong serialization.
- **Relational Integrity:** Foreign key constraints prevent orphaned transaction records, invalid account references, or unlinked ledger entries.
- **Complex Multi-Table Joins & Regulatory Audits:** Financial audits require multi-table reconciliation queries which are natural in SQL with indexed joins.
- **Where NoSQL IS used in Banking:** Storing session states (Redis), audit log ingestion, clickstream analytics, or product catalog lookup (DynamoDB/MongoDB).

Q8. What is Optimistic vs. Pessimistic Locking? When would you use `SELECT ... FOR UPDATE`?

- **Pessimistic Locking (`SELECT ... FOR UPDATE`):** Locks the selected account row immediately until the transaction commits or rolls back. Other concurrent transactions must wait.

Bank Use Case: ATM cash withdrawal or balance debit where race conditions cannot be tolerated.

- **Optimistic Locking:** Does not lock the row during reading. Instead, it checks a ``version`` column before updating:
`UPDATE accounts SET balance = 500, version = version + 1 WHERE id = 101 AND version = 3;`
If ``rows_affected` == 0`, a conflict occurred and the application retries. Best for read-heavy systems with low collision frequency.

Q9. How does Database Indexing work internally, and why shouldn't you index every column?

- **B+ Tree Indexing:** Balanced tree data structure allowing $O(\log N)$ search, insertion, and range queries. Leaf nodes are linked as a doubly-linked list for sequential scans.
- **Write Penalty on High-Frequency Tables:** In high-throughput banking transaction tables, every ``INSERT`` or ``UPDATE`` forces the database engine to update all associated index trees and rebalance nodes, degrading write performance significantly.

4 System Design & Security Scenarios

Q10. System Design Scenario: How would you architect DBBL's ATM backend to handle 10x traffic surges during Eid without crashing or double-debiting?

Architectural Blueprint:

1. **Read/Write Separation:** Direct account balance inquiries and mini-statement reads to *Read Replicas* or a fast Redis cache cluster. Reserve the *Primary Master Database* strictly for financial write transactions.
2. **Connection Pooling:** Use database connection poolers (like PgBouncer / HikariCP) to prevent thousands of ATM terminals from overwhelming database thread limits.
3. **Asynchronous Non-Critical Pipelines:** Offload SMS alerts, email receipts, push notifications, and analytics to a message broker (Kafka / AWS SQS / RabbitMQ).
4. **Circuit Breaker Pattern:** If downstream core banking services experience latency, circuit breakers (e.g. Resilience4j) fail fast with graceful user messaging instead of hanging ATM hardware.

Q11. What is the difference between Symmetric vs. Asymmetric Encryption, and how are they used in banking?

- **Symmetric Encryption (AES-256):** Same secret key for both encryption and decryption. Very fast. Used for encrypting database data-at-rest and ATM PIN blocks (in HSM - Hardware Security Modules).
- **Asymmetric Encryption (RSA / ECC):** Public key encrypts, private key decrypts. Slower. Used for TLS/SSL handshake, digital signatures, and securely exchanging the symmetric session key.
- **Hashing (bcrypt / Argon2 / SHA-256):** One-way cryptographic transformation with salt. Used for storing passwords and PIN verification.

5 Machine Learning & Research Alignment

Q12. You have 3 published AI/ML papers. How can machine learning specifically benefit DBBL IT operations?

- **Real-Time Fraud & Anomaly Detection:** Train classification models on historical transaction vectors (time, amount, velocity, geocoding) to flag suspicious ATM or card transactions in milliseconds.
- **e-KYC & Biometric Verification:** Your research in facial attribute estimation directly maps to digital customer onboarding in Rocket and NexusPay (matching NID photo with live selfie and liveness detection).
- **ATM Cash Demand Forecasting:** Using time-series forecasting models (ARIMA / LSTM / Random Forest — as explored in your published drought forecasting paper) to predict daily cash depletion per ATM and optimize armored cash replenishment logistics.

6 Behavioral & HR Questions

Q13. You are currently working in a software development agency (XORGeek). Why do you want to switch to a bank like DBBL?

Winning Angle: "At XORGeek, I have proven my ability to build and deploy 6 live production systems across mobile, web, and cloud. However, I want to apply my engineering skills to mission-critical, large-scale financial infrastructure where reliability, zero downtime, and data integrity impact millions of citizens daily. DBBL is the pioneer in ATM networks, FastTrack, and mobile banking in Bangladesh, offering the exact scale and high-stake engineering environment I want to build my long-term career in."

Q14. Banking IT requires 24/7 on-call availability, night maintenance windows, and emergency troubleshooting. Are you prepared for this?

Winning Angle: "Yes, absolutely. In my current role, managing live production deployments and race-day live tracking for RunMate Club already demanded off-hour monitoring, high-stress issue resolution, and immediate hotfixes. I understand that banking services operate 24/7/365, and I am fully committed to rotational on-call and weekend maintenance schedules."

7 Rapid 5-Minute Pre-Viva Checklist

Technical Terms to Say Confidently:

- ACID Transactions & Two-Phase Commit (2PC)

DBBL Facts to Keep in Mind:

- Largest ATM & FastTrack network in Bangladesh

- Database Indexing (B+ Tree, Clustered vs Non-Clustered)
- Maker-Checker Workflow & Tamper-proof Audit Logs
- Idempotency Keys & Server-side IPN Handshakes
- Core Products: Rocket (MFS), NexusPay, Agent Banking
- Switch Networks: NPSB, VISA, MasterCard, UnionPay
- Core Banking Solution (CBS): High-availability Oracle/Unix backend